



تمرین چهار

برنامه‌سازی پیشرفته

دانشکده علوم مهندسی

طراح تمرین: سروش نصیری و ایلینا افضلی

بهار ۱۴۰۵



۱ مقدمه

ایلیا عاشق موسیقی است. هزاران فایل صوتی روی هارددیسکش دارد که هیچ سازمان‌دهی‌ای ندارند. هیچ پخش‌کننده‌ای پیدا نکرده که هم رایگان باشد، هم روی لپ‌تاپ قدیمی‌اش درست کار کند. تصمیم می‌گیرد با دانشی که در درس برنامه‌نویسی پیشرفته کسب کرده، پخش‌کننده‌ی خودش را بنویسد؛ برنامه‌ای که کاملاً درون ترمینال اجرا می‌شود.

وظیفه‌ی شما طراحی و پیاده‌سازی یک **پخش‌کننده‌ی موسیقی ترمینالی** به زبان C++ است. برنامه باید در محیط CLI/TUI اجرا شود – هیچ فریم‌ورک گرافیکی مجاز نیست. این پروژه فرصتی است تا طیف گسترده‌ای از مهارت‌های برنامه‌نویسی پیشرفته را در یک بستر واقعی نشان دهید.

۲ اهداف یادگیری

مباحث زیر را مرور خواهید کرد:

- طراحی یک برنامه‌ی C++ چندکلاسه و چندفایله از صفر
- اعمال اصول OOP: کپسوله‌سازی، وراثت، چندریختی
- مدیریت حافظه‌ی پویا و جلوگیری از نشت حافظه
- خواندن از فایل‌های CSV و M3U برای بارگذاری متادیتا و playlist ها
- کار با کانتینرهای STL و الگوریتم‌های استاندارد
- مدیریت خطاهای زمان اجرا و ورودی‌های نامعتبر کاربر
- ساخت رابط‌های ترمینالی تعاملی و منوگرا

۳ نمای کلی قابلیت‌ها

📁 کتابخانه و Playlist ها

- بارگذاری متادیتا از CSV مرکزی
- switch بین playlist های M3U داده‌شده
- filter بر اساس album یا artist

🎵 موتور پخش (Playback Engine)

- play, pause, stop, resume
- next / previous
- حالت‌های shuffle و repeat

🖥️ رابط ترمینال (TUI)

- Now Playing
- Playlists
- Browse Playlist
- Settings

📦 پیکربندی (Config)

- ذخیره‌ی آخرین آهنگ پخش‌شده
- ذخیره مود فعلی پخش آهنگ ها
- ذخیره ی فعال



۴ ساختار پوشه‌ی پروژه

ساختار پوشه‌ی پروژه از پیش مشخص است. برنامه باید داده‌های خود را از مسیرهای ثابت زیر بخواند – نیازی به دریافت مسیر از خط فرمان نیست.

```
Music_Player/
├── src/
│   ├── miniaudio.h      ← کتابخانه‌ی صوتی (از پیش داده شده)
│   ├── ...               ← (h و .cpp) فایل‌های پیاده‌سازی شما
├── AP_CA4.pdf            ← صورت‌سؤال تمرین (این فایل)
├── Data/
│   ├── library.csv       ← متادیتای تمام آهنگ‌ها
│   ├── settings.cfg      ← تنظیمات ذخیره شده (توسط برنامه ساخته می‌شود)
│   ├── Playlists/
│   │   └── *.m3u         ← playlist فایل‌های
│   ├── Musics/
│   │   └── *.mp3         ← فایل‌های موسیقی
```

مسیرهایی که برنامه به صورت ثابت از آن‌ها می‌خواند:

منبع داده	مسیر نسبی به ریشه‌ی پروژه
متادیتای آهنگ‌ها	Data/library.csv
تنظیمات برنامه	Data/settings.cfg
فایل‌های Playlist	Data/Playlists/
فایل‌های موسیقی	Data/Musics/

نکته – miniaudio.h

فایل miniaudio.h در پوشه‌ی src/ از پیش داده شده است. این فایل را تغییر ندهید. بقیه‌ی فایل‌های پیاده‌سازی (h و .cpp) را خودتان در همین پوشه بسازید.

نکته – مسیر فایل‌های M3U

مسیرهای درج شده در فایل‌های .m3u باید با فیلد file_path در library.csv تطابق داشته باشند. توصیه می‌شود مسیرهای .m3u را نسبی بنویسید (مثلاً Data/Musics/song.mp3) تا پروژه روی هر سیستمی قابل اجرا باشد.



۵ مدل داده – CSV و M3U

۵.۱ فایل CSV متادیتا

تمام متادیتای آهنگ‌ها در یک فایل CSV مرکزی ارائه می‌شود – نه از تگ‌های ID3 و نه با ورود دستی کاربر. هر سطر یک آهنگ است. ساختار انتظاری:

```
# library.csv – هر سطر یک آهنگ
title,artist,album,genre,year,duration_sec,file_path
Bohemian Rhapsody,Queen,A Night at the Opera,Rock,1975,354,/music/bohemian.mp3
Hotel California,Eagles,Hotel California,Rock,1977,391,/music/hotel.mp3
```

برنامه در زمان راه‌اندازی این فایل را یک‌بار می‌خواند و تمام آهنگ‌ها را در `MusicLibrary` بارگذاری می‌کند. اگر فیلدی خالی یا نادرست باشد، آن سطر با یک هشدار رد می‌شود و برنامه ادامه می‌یابد.

۵.۲ فایل‌های M3U (playlist ها)

هر playlist یک فایل `.m3u` است که مسیر فایل‌های صوتی را فهرست می‌کند. تمام مسیرهای موجود در M3U باید زیرمجموعه‌ای از آهنگ‌های CSV باشند – تطابق از طریق `file_path` انجام می‌شود. برنامه هنگام راه‌اندازی همه‌ی فایل‌های `.m3u` موجود در یک پوشه‌ی مشخص را بارگذاری می‌کند.

```
# rock_hits.m3u
/music/bohemian.mp3
/music/hotel.mp3
/music/stairway.mp3
```

نکته – playlist های ثابت

playlist ها فقط خواندنی هستند. کاربر نمی‌تواند آهنگی اضافه، حذف، یا ذخیره کند. تنها عمل مجاز روی playlist ها `switch` بین آن‌هاست. اگر فایل M3U حاوی مسیری باشد که در CSV نیست، آن مسیر نادیده گرفته می‌شود.

۵.۳ فیلدهای Song

فیلد	نوع	توضیح
<code>title</code>	string	نام آهنگ
<code>artist</code>	string	نام هنرمند یا گروه
<code>album</code>	string	نام آلبوم
<code>genre</code>	string	ژانر موسیقی
<code>year</code>	int	سال انتشار (چهاررقمی)
<code>duration</code>	int (ثانیه)	مدت زمان؛ نمایش به صورت <code>MM:SS</code>
<code>filePath</code>	string	مسیر نسبی فایل صوتی



۶ معماری نرم‌افزار و طراحی کلاس‌ها

پروژه باید در یک ساختار چندلایه پیاده‌سازی شود تا ابتدا منطق اصلی تکمیل شود و سپس قابلیت‌های UI اضافه گردند. این رویکرد امکان تست مستقل هر لایه را فراهم می‌کند.

۶.۱ کلاس‌های اصلی (پیشنهادی)

کلاس	مسئولیت	اعضای کلیدی
Song	یک آهنگ با تمام متادیتایش	album, artist, title filePath, duration
Playlist	مجموعه‌ای مرتب از اشاره‌گرهای Song	songs (vector of pointers), name
MusicLibrary	کاتالوگ کل آهنگ‌ها؛ جستجو و filter در اینجا پیاده میشوند.	filterByArtist(), getSong(), filterByAlbum()
CsvLoader	خواندن library.csv و پر کردن MusicLibrary	load(path, library)
M3uLoader	خواندن فایل‌های .m3u از پوشه	loadAll(dir, library)
Player	ماشین حالت پخش: play/pause/stop/resume/next/prev	mode, state, currentSong
ConfigManager	بارگذاری و ذخیره‌ی settings.cfg	set(key, value), get(key) load()
Screen (abstract)	رابط مشترک همه‌ی صفحات	virtual void render() = 0 handleInput()
UIRenderer	تمام خروجی ترمینال را متمرکز مدیریت می‌کند؛	println(), drawBox(), clearScreen()
InputHandler	تمام ورودی کاربر را دریافت و اعتبارسنجی می‌کند؛ جداسازی کامل از منطق برنامه	readInt(min, max), readLine(), readKey()
Application	کنترلر سطح بالا؛ مالک همه‌ی زیرسیستم‌ها	shutdown(), run(), init()

نکته‌ی طراحی – جداسازی نگرانی‌ها

کلاس‌های پایه‌ای مثل Playlist و Song نباید مستقیماً std::cout صدا بزنند یا از std::cin بخوانند. تمام خروجی ترمینال از طریق کلاس UIRenderer و تمام ورودی از طریق کلاس InputHandler انجام می‌شود. این دو کلاس در لایه‌های نهایی قرار دارند و توسط Screen های مشتق‌شده استفاده می‌شوند. این جداسازی امکان تست لایه‌های پایین‌تر بدون UI را فراهم می‌کند.



۷ جزئیات قابلیت‌های موردنیاز

۷.۱ بارگذاری داده

- مسیرهای داده ثابت هستند (طبق ساختار بخش ۴):
 - از CSV از `Data/library.csv` و playlist ها از `Data/Playlists/` بارگذاری می‌شوند.
- `CsvLoader` فایل CSV را پارس می‌کند و اشیاء `Song` را در `MusicLibrary` ذخیره می‌کند.
- `M3uLoader` تمام فایل‌های `.m3u` پوشه‌ی `Data/Playlists/` را می‌خواند و یک `Playlist` به ازای هر فایل می‌سازد.
- اگر مسیری در M3U در CSV نباشد، آن سطر نادیده گرفته می‌شود.

۷.۲ کنترل‌های پخش (Playback Controls)

- play** – شروع پخش آهنگ جاری از ابتدا.
- pause** – متوقف کردن پخش؛ موقعیت حفظ می‌شود.
- resume** – ادامه از موقعیت متوقف‌شده.
- stop** – توقف پخش و بازنشانی موقعیت به صفر.
- next** – رفتن به آهنگ بعدی بر اساس حالت پخش فعال.
- previous** – رفتن به آهنگ قبلی.

۷.۳ حالت‌های پخش (Playback Modes)

حالت	رفتار پس از پایان آهنگ
NO_REPEAT	رفتن به آهنگ بعدی. توقف پس از آخرین آهنگ.
REPEAT_ONE	تکرار بی‌نهایت همان آهنگ.
REPEAT_ALL	loop کردن کل playlist از ابتدا.
SHUFFLE	انتخاب تصادفی آهنگ بعدی (بدون تکرار فوری).

۷.۴ Switch بین Playlist ها

- کاربر می‌تواند از منوی playlist ها یکی را انتخاب کند.
- با انتخاب یک playlist جدید، پخش جاری stop می‌شود و playlist فعال تغییر می‌کند.
- playlist ها ثابت و فقط خواندنی‌اند – ویرایش یا ذخیره‌ی آن‌ها توسط کاربر امکان‌پذیر نیست.

۷.۵ Filter بر اساس Artist یا Album

کاربر می‌تواند لیست آهنگ‌های playlist فعال را بر اساس نام artist یا album فیلتر کند. منویی از مقادیر موجود (هنرمندان یا آلبوم‌های موجود در playlist) به کاربر نشان داده می‌شود تا یکی را انتخاب کند. نتیجه‌ی filter، آهنگ‌های منطبق را در یک لیست قابل پخش نمایش می‌دهد.



۷.۶ ذخیره‌ی آخرین آهنگ

هنگام خروج از برنامه (quit)، ConfigManager باید مسیر فایل آخرین آهنگ پخش‌شده را در settings.cfg ذخیره کند. در اجرای بعدی، این آهنگ در صفحه‌ی Now Playing نمایش داده می‌شود تا کاربر بداند آخرین بار کجا بوده – پخش خودکار نمی‌شود اما آهنگ آماده است.

۷.۷ مرتب‌سازی (Sort)

در صفحه‌ی **Playlist View**، کاربر می‌تواند آهنگ‌های playlist را مرتب کند. مرتب‌سازی صرفاً روی نمایش اعمال می‌شود – ترتیب اصلی playlist تغییر نمی‌کند. گزینه‌های مرتب‌سازی:

- بر اساس **Title** (الفبایی)
 - بر اساس **Artist** (الفبایی)
 - بر اساس **Album** (الفبایی)
 - بر اساس **Year** (صعودی / نزولی)
 - بر اساس **Duration** (کوتاه‌ترین به طولانی‌ترین)
- استفاده از std::sort با lambda مناسب است.

پایداری: مرتب‌سازی فقط تا زمانی که کاربر در Playlist View است اعمال می‌شود. با خروج از این صفحه (کلید [0])، ترتیب به حالت پیش‌فرض (ترتیب اصلی playlist) بازمی‌گردد.

۷.۸ جستجو (Search)

کاربر می‌تواند در صفحه‌ی **Playlist View** یک رشته وارد کند. برنامه آهنگ‌هایی را نمایش می‌دهد که رشته‌ی جستجو در عنوان و یا هنرمند آن‌ها وجود داشته باشد (case-insensitive). نتیجه در همان صفحه نمایش داده می‌شود و قابل پخش است. برای پاک کردن جستجو کاربر رشته‌ی خالی وارد می‌کند.

پایداری: با خروج از Playlist View (کلید [0])، جستجوی فعال پاک می‌شود و در بازگشت همه‌ی آهنگ‌ها نمایش داده می‌شوند.

نکته – فرمت settings.cfg

از یک فرمت ساده‌ی key=value استفاده کنید. مثال:

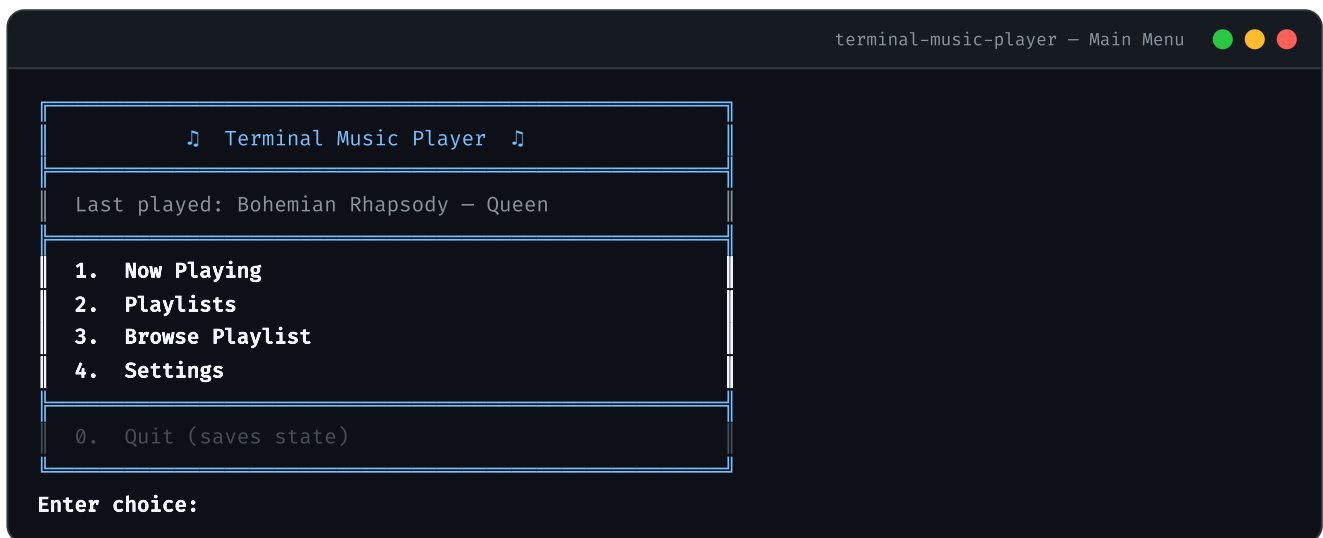
```
active_playlist=rock_hits
playback_mode=SHUFFLE
last_song=/music/queen/bohemian_rhapsody.mp3
```



۸ رابط کاربری ترمینال (TUI)

برنامه از پنج صفحه تشکیل شده است. از هر صفحه‌ای با [0] یا [q] می‌توان برگشت. quit فقط از Main Menu قابل اجراست و قبل از خروج، وضعیت را در settings.cfg ذخیره می‌کند.

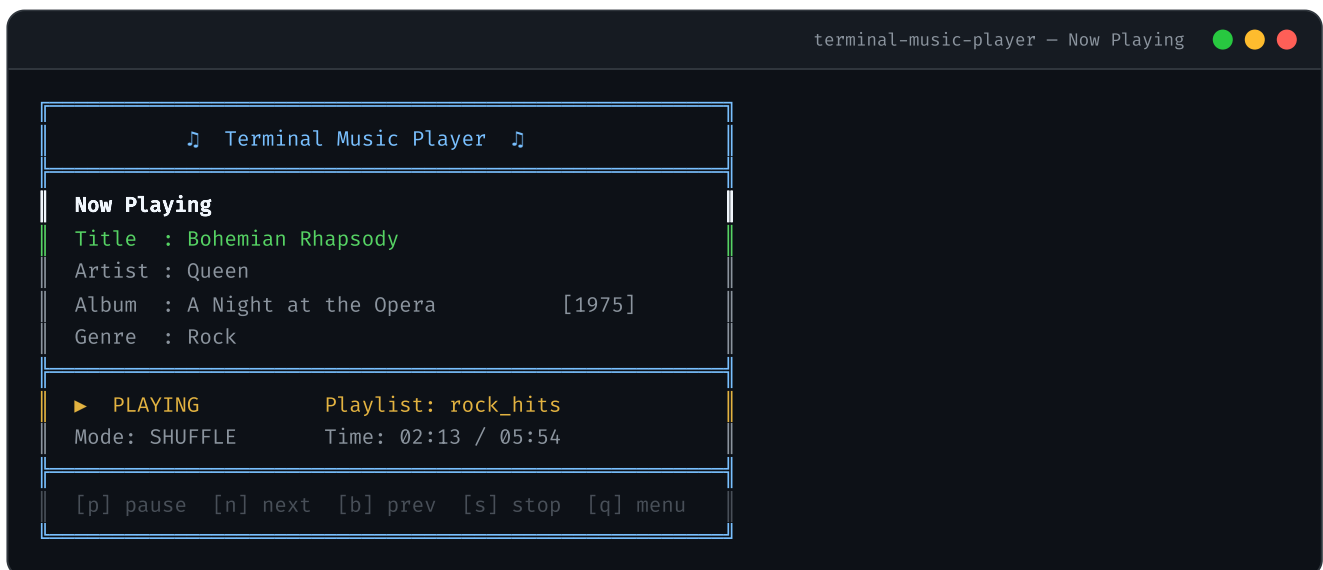
۸.۱ Main Menu



سطر «Last played» آخرین آهنگ ذخیره‌شده در settings.cfg را نشان می‌دهد. اگر هنوز هیچ آهنگی پخش نشده، این سطر نمایش داده نمی‌شود.

۸.۲ صفحه‌ی Now Playing

ورود: گزینه‌ی 1 از Main Menu. خروج: کلید [q].

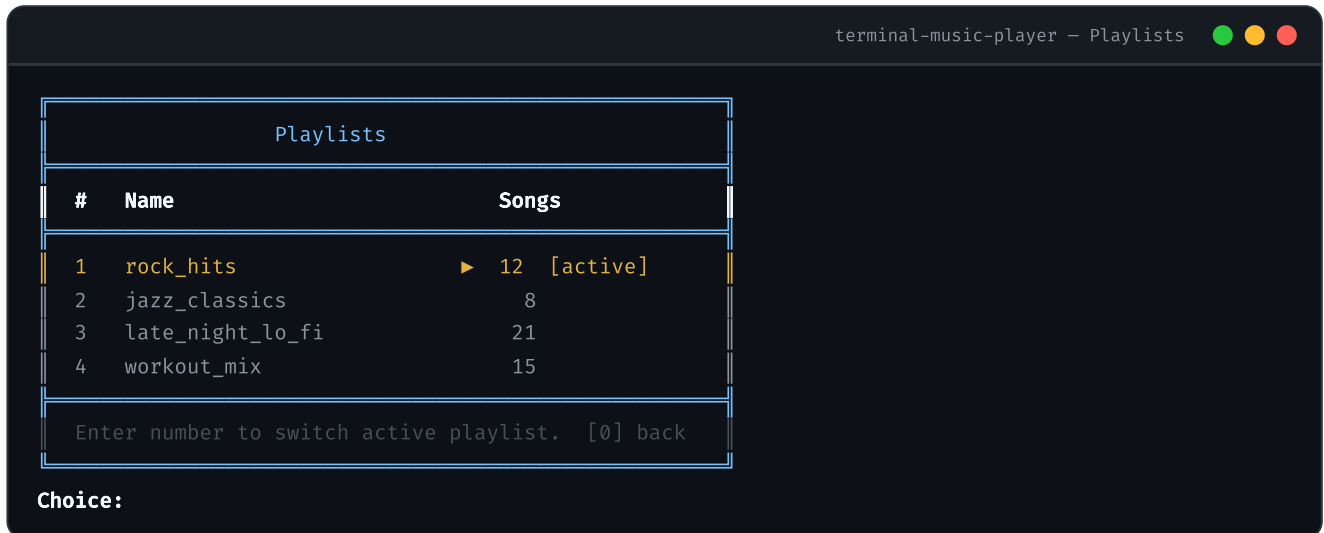


سطر «Time» زمان گذشته و مدت کل آهنگ را به فرمت MM:SS / MM:SS نشان می‌دهد. نوار پیشرفت وجود ندارد – فقط اعداد.



۸.۳ صفحه‌ی Playlist List

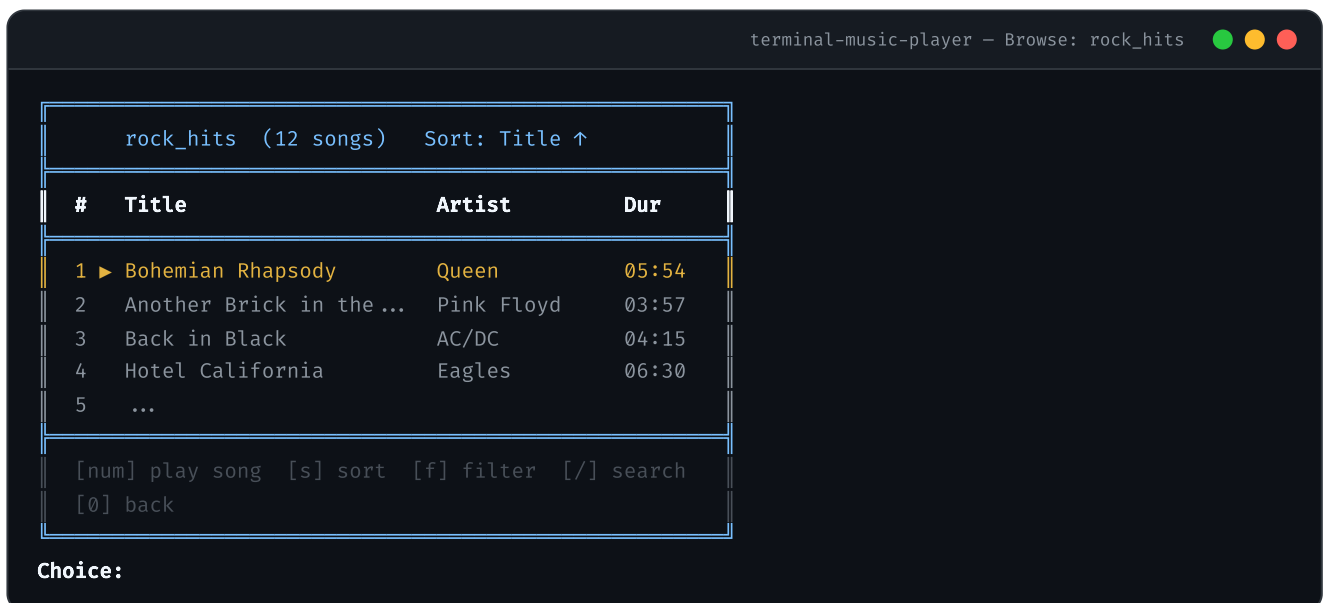
ورود: گزینه‌ی 2 از Main Menu. خروج: عدد 0.



کاربر عدد playlist را تایپ می‌کند. اگر عددی غیر از playlist فعال انتخاب شود، پخش جاری متوقف و playlist فعال عوض می‌شود. آهنگ جدیدی شروع نمی‌شود – کاربر باید خودش به Now Playing برود و play کند.

۸.۴ صفحه‌ی Playlist View (Browse)

این صفحه مرکز تعامل با محتوای playlist است. از همینجا می‌توان آهنگ انتخاب کرد، sort کرد، search کرد، و filter.



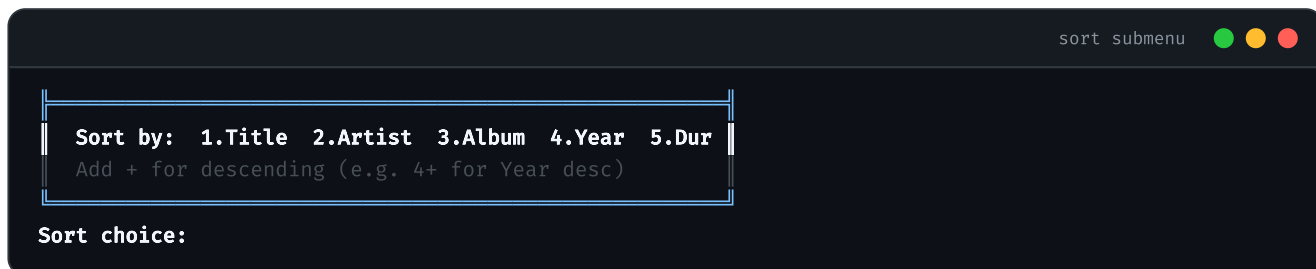
ورودی‌های ممکن در این صفحه:

- عدد (مثلاً 3) – آهنگ شماره‌ی ۳ را play می‌کند و به صفحه‌ی Now Playing می‌رود.
- s – زیرمنوی Sort را باز می‌کند (همین صفحه، بدون رفتن جای دیگر).
- f – به صفحه‌ی Filter می‌رود.
- / – حالت جستجو را فعال می‌کند (prompt برای تایپ کلمه).
- 0 – برگشت به Main Menu.



۸.۵ Sort در Playlist View

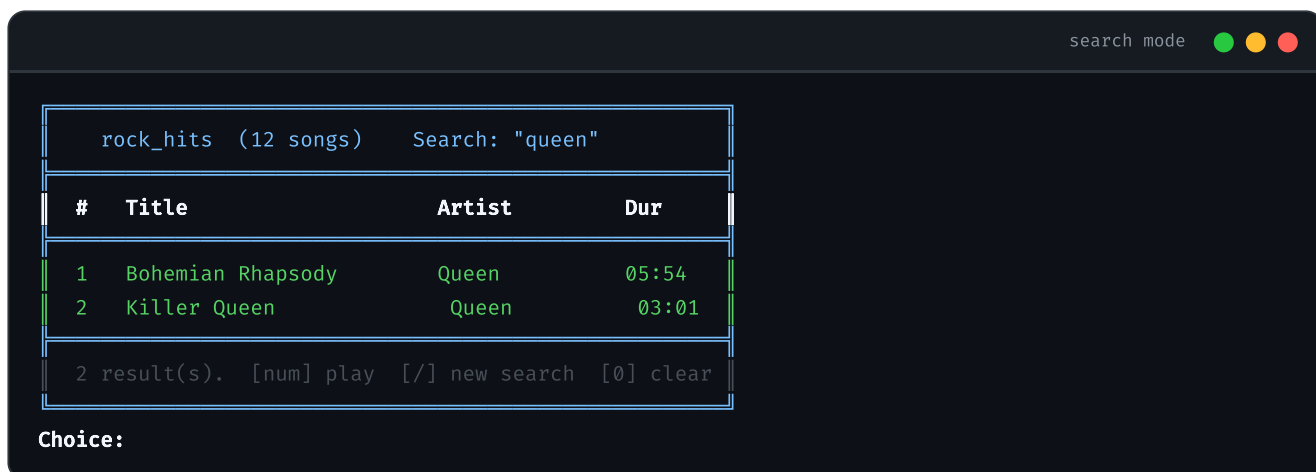
وقتی کاربر می‌زند، یک زیرمنوی کوچک پایین صفحه ظاهر می‌شود:



پس از انتخاب، همان Playlist View بلافاصله با ترتیب جدید refresh می‌شود. مرتب‌سازی تنها روی نمایش تأثیر می‌گذارد – فایل دست‌نخورده می‌ماند.

۸.۶ Search در Playlist View

وقتی کاربر می‌زند، prompt تغییر می‌کند:



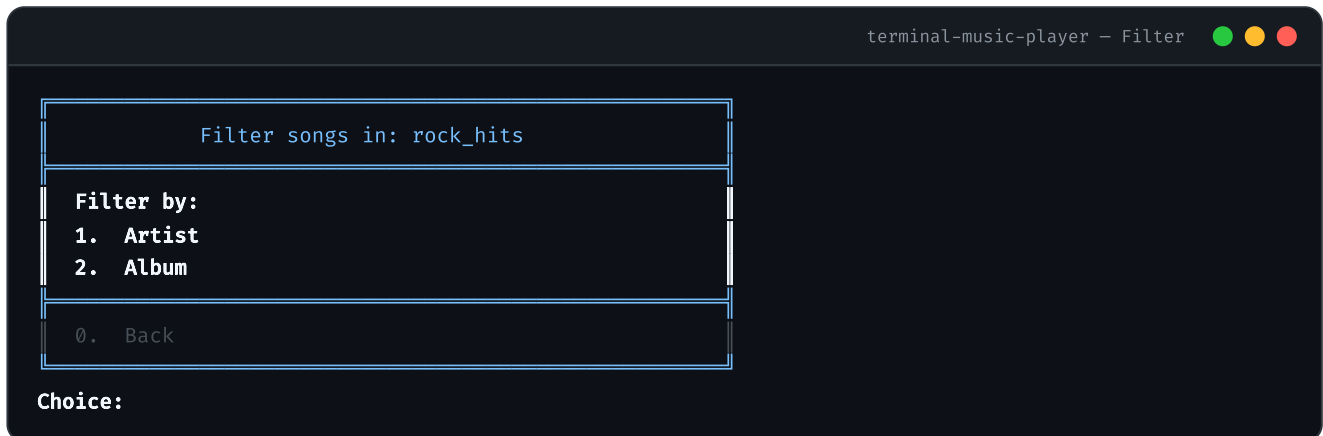
جستجو در **عنوان، هنرمند، و آلبوم** همزمان انجام می‌شود (case-insensitive). برای پاک کردن جستجو، در این حالت همه‌ی آهنگ‌ها را دوباره نشان می‌دهد. نتایج جستجو نیز قابل پخش هستند.



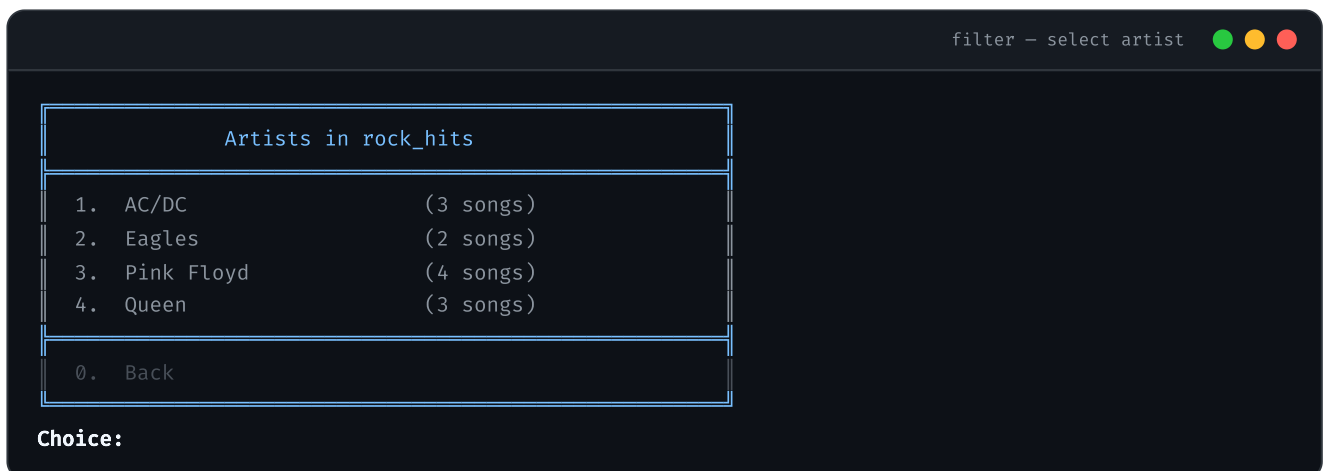
۸.۷ صفحه‌ی Filter

ورود: کلید `f` از Playlist View. خروج: عدد `0` (برمی‌گردد به Playlist View).

کاربر ابتدا نوع فیلتر را انتخاب می‌کند (Artist یا Album)، سپس از لیست مقادیر موجود در playlist یکی را انتخاب می‌کند. نتیجه همان‌جا نمایش داده می‌شود.



پس از انتخاب `1` (Artist)، لیست هنرمندان موجود در playlist نمایش داده می‌شود:



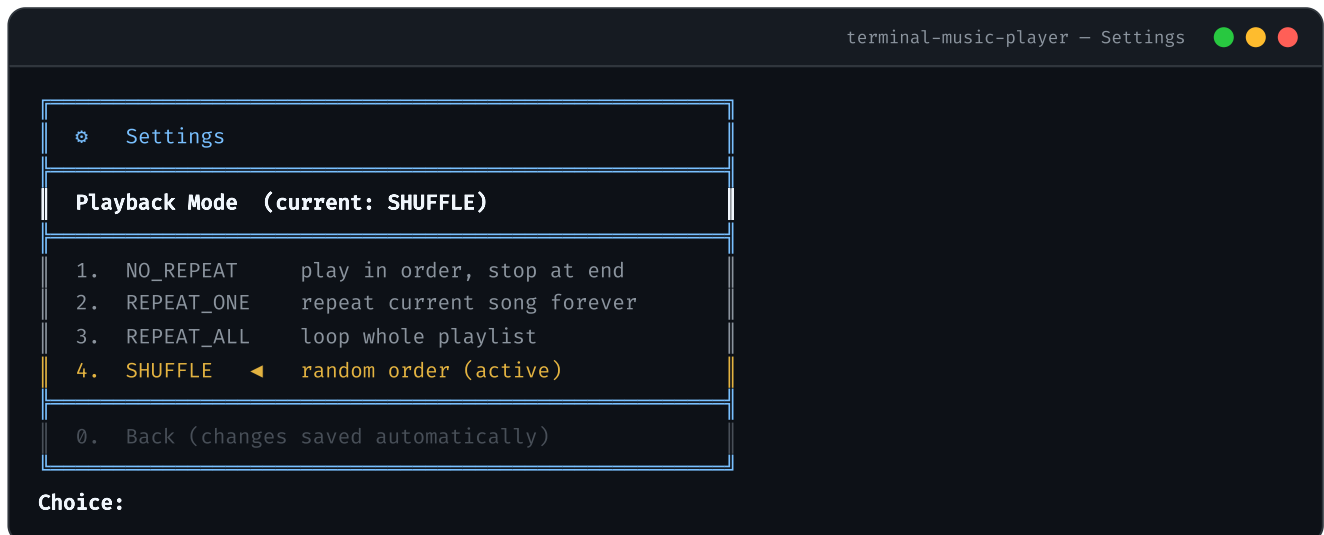
پس از انتخاب هنرمند، همان Playlist View با فقط آهنگ‌های آن هنرمند نمایش داده می‌شود و قابل پخش است. کاربر با زدن `[0]` در این حالت مستقیماً به Playlist View کامل (بدون فیلتر) بازمی‌گردد – نه به صفحه‌ی Filter. برای اعمال فیلتر جدید، دوباره `[f]` بزنید.



۸.۸ صفحه‌ی Settings

ورود: گزینه‌ی 4 از Main Menu. خروج: عدد 0.

از این صفحه حالت پخش (Shuffle / Repeat) تنظیم می‌شود. انتخاب بلافاصله اعمال می‌شود و در settings.cfg ذخیره می‌گردد.



پس از انتخاب، برنامه تأیید می‌دهد و صفحه‌ی Settings را با مقدار به‌روزشده دوباره نمایش می‌دهد. کاربر با 0 به Main Menu برمی‌گردد.

۸.۹ مدیریت ورودی

- **ورودی منو:** اعداد یا حروف از طریق `std::cin` وارد می‌شود. ورودی خارج از محدوده پیام خطا می‌دهد و دوباره می‌پرسد – هرگز crash نمی‌کند.
- **ورودی آزاد (search):** فاصله‌های ابتدا/انتهای حذف می‌شوند. رشته‌ی خالی جستجو را پاک می‌کند.

۸.۱۰ آزادی در طراحی رابط کاربری

نکته

تصاویر و نمونه‌های نمایش‌داده‌شده در این بخش صرفاً جهت نمونه و ایده‌دهی هستند و پیاده‌سازی دقیقاً مطابق آن‌ها الزامی نیست. شما آزاد هستید رابط کاربری ترمینال را با سلیقه و خلاقیت خود طراحی کنید – به شرط آنکه تمام قابلیت‌های خواسته‌شده به‌درستی پیاده‌سازی شده باشند.



۹ مدیریت خطا

برنامه‌ی شما باید در برابر تمام شرایط خطا و ورودی‌های نامعتبر مقاوم باشد. هیچ ورودی یا وضعیتی نباید باعث crash شود. جداول زیر خطاهای اصلی و رفتار مورد انتظار را مشخص می‌کنند.

۹.۱ خطاهای ورودی کاربر

ورودی نامعتبر	رفتار مورد انتظار
عدد خارج از محدوده‌ی منو (مثلاً ۹ در منویی با ۴ گزینه)	پیام "Invalid choice. Please try again." و prompt تکرار می‌شود.
ورودی غیرعددی جایی که عدد انتظار می‌رود (مثلاً abc)	cin پاکسازی می‌شود، پیام خطا چاپ و prompt تکرار می‌شود.
شماره‌ی آهنگ در Playlist View خارج از محدوده است	پیام "No song at this index." و prompt تکرار می‌شود.
رشته‌ی خالی در جستجو (search)	همه‌ی آهنگ‌ها دوباره نمایش داده می‌شوند.
تلاش برای play وقتی playlist خالی است	پیام "Playlist is empty." نمایش داده می‌شود.

۹.۲ خطاهای پخش صوت

وضعیت خطا	رفتار مورد انتظار
فایل صوتی در مسیر مشخص‌شده وجود ندارد	پیام خطا نمایش داده می‌شود و به آهنگ بعدی رفته می‌شود.
تلاش برای pause یا resume بدون آهنگ فعال	دستور بی‌صدا نادیده گرفته می‌شود – crash نمی‌کند.

⚠ پاکسازی std::cin بعد از ورودی نامعتبر

وقتی کاربر رشته‌ای غیرعددی وارد کند، std::cin وارد حالت fail می‌شود. پس از هر شکست خواندن حتماً cin.clear() و سپس cin.ignore() را فراخوانی کنید – در غیر این صورت حلقه‌ی ورودی بی‌نهایت تکرار می‌شود بدون اینکه چیزی از کاربر بخواند.



۱۰ الزامات OOP و فنی

۱۰.۱ مفاهیم OOP اجباری

مفهوم	محل اعمال
Encapsulation	تمام اعضای داده باید <code>private</code> یا <code>protected</code> باشند. دسترسی فقط از طریق <code>getter/setter</code> یا متدهای معنادار عمومی.
Inheritance	حداقل یک سلسله‌مراتب کلاسی با دو سطح (مثلاً <code>abstract Screen</code> → کلاس‌های صفحه‌ی مشتق‌شده).
Polymorphism	حداقل یک متد <code>virtual</code> که از طریق یک اشاره‌گر به کلاس پایه فراخوانی می‌شود. <code>destructor</code> کلاس‌های پایه باید <code>virtual</code> باشد.
Abstraction	متد <code>pure virtual</code> (مثلاً <code>virtual void render() = 0</code>) در <code>Screen</code> .
Dynamic Memory	استفاده‌ی صریح از <code>new</code> / <code>delete</code> در حداقل یک کلاس، یا استفاده از <code>smart pointer</code> ها با توضیح مدل مالکیت.
File I/O	<code>std::ofstream</code> / <code>std::ifstream</code> برای CSV، M3U، و <code>settings.cfg</code> .
STL Containers	استفاده از <code>container</code> ها مانند <code>(std::vector, std::map)</code> .
Error Handling	استفاده از <code>try</code> / <code>catch</code> برای خطاهای I/O. اعتبارسنجی ورودی‌های کاربر.

۱۰.۲ الزامات کیفیت کد

- هر کلاس در فایل‌های `.h` و `.cpp` جداگانه. `header` ها باید `include guard` یا `#pragma once` داشته باشند.
- اصول `Clean Code` رو رعایت کنید.
- هیچ متغیر `global` قابل‌تغییری وجود نداشته باشد.

⚠ سیاست نشت حافظه

هر تخصیص `heap` باید یک مسیر آزادسازی متناظر داشته باشد. `MusicLibrary` تنها مکان معتبر برای آزادسازی اشیاء `Song` است؛ `destructor` کلاس‌های `Playlist` نباید آهنگ‌ها را حذف کند.



۱۱ قابلیت‌های اختیاری (Bonus)

قابلیت‌های زیر اختیاری‌اند و نمره‌ی اضافه می‌آورند. هر کدام مستقل هستند.

نمره	توضیح	Bonus
5+	استفاده از <code>ncurses</code> / <code>termios</code> برای دریافت ضربه‌ی کلید بدون نیاز به <code>Enter</code> .	Raw Keyboard Input
5+	پایده‌سازی کل UI با <code>ncurses</code> : هایلایت رنگی سطر فعال.	Full ncurses TUI
5+	ردیابی تعداد دفعات پخش هر آهنگ؛ playlist خودکار «پرتکرارترین آهنگ‌ها».	Play History
5+	امکان علامت‌گذاری آهنگ‌ها به عنوان موردعلاقه (♥) با playlist اختصاصی.	Favourites System
5+	کاربر می‌تواند از میان آهنگ‌های موجود در <code>library.csv</code> یکی را انتخاب کرده و به یک فایل <code>.m3u</code> دلخواه اضافه کند. تغییر باید بلافاصله روی دیسک نوشته شود و playlist در حافظه بروزرسانی گردد.	افزودن آهنگ به Playlist
5+	جابجایی موقعیت پخش با کلیدهای ← → در صفحه‌ی Now Playing (هر بار ± ۱۰ ثانیه). راهنمای پایده‌سازی در بخش ۱۳ آمده است.	Seek

نکته – Bonus کلیدهای جهت‌نما

قابلیت seek با کلیدهای ← → در صفحه‌ی Now Playing نیاز به دریافت کلیدهای special دارد که با `std::cin` معمولی ممکن نیست. برای پایده‌سازی آن باید terminal را در حالت raw قرار دهید (از طریق `termios` یا `ncurses`). این موضوع با «Raw Keyboard Input» گره خورده است.

راهنمای Bonus افزودن آهنگ به Playlist

فقط آهنگ‌هایی که در `library.csv` موجودند مجاز به اضافه شدن هستند – مسیرهای خارجی پذیرفته نمی‌شوند. برای نوشتن روی دیسک کافی است `filePath` آهنگ را با `std::ofstream` در حالت `std::ios::app` به انتهای فایل `.m3u` اضافه کنید. سپس اشاره‌گر آن `Song` را نیز به `Playlist` در حافظه اضافه کنید تا UI بدون نیاز به restart همگام بماند.



۱۲ راهنمایی‌ها و سؤالات متداول

💡 از کجا شروع کنم؟ – ترتیب پیشنهادی توسعه

UI را آخر بسازید. یک backend کارا بدون UI بسیار ارزشمندتر از یک منوی زیبایی است که crash می‌کند.

1. `Song.h` + `Song.cpp` – ساده‌ترین کلاس، فوری قابل تست
2. `CsvLoader` – پارس CSV و ساخت اشیاء `Song`
3. `MusicLibrary` – نگهداری و فیلتر کردن `Song` ها
4. `M3uLoader` + `Playlist` – بارگذاری فایل‌های `.m3u`
5. `Player` – ماشین حالت (ابتدا شبیه‌سازی بدون صوت واقعی)
6. `ConfigManager` – ذخیره و بارگذاری `settings.cfg`
7. `UIRenderer` + `InputHandler` – کلاس‌های پایه‌ی UI
8. صفحات مشتق‌شده از `Screen` – هر صفحه جداگانه کار میکند
9. `Application` – حلقه‌ی اصلی که همه را به هم متصل می‌کند

🚩 آیا باید صوت واقعی پخش کنم؟

بله. پخش صوت واقعی از طریق `miniaudio` الزامی است. با این حال توصیه می‌شود ابتدا معماری و منطق کنترل پخش را پیاده‌سازی کنید، سپس `miniaudio` را یکپارچه کنید. راهنمای کامل در بخش ۱۳ موجود است.

💡 چگونه مدت زمان را نمایش دهم؟

مدت زمان را به صورت تعداد ثانیه ذخیره کنید. برای نمایش: `int mins = dur / 60; int secs = dur % 60;` و با `std::setw(2) << std::setfill('0')` با صفر پر کنید (مثلاً `03:07`).

💡 چگونه ترمینال را پاک کنم؟

می‌توانید از `system("clear")` روی Linux و یا `system("cls")` بر روی Windows استفاده کنید.



تراز کردن ستون‌ها با `std::setw`

`std::setw(n)` از هدر `<iomanip>`، عرض حداقل فیلد بعدی خروجی را روی `n` کاراکتر تنظیم می‌کند. اگر متن کوتاه‌تر باشد، با فاصله پر می‌شود. این ابزار برای نمایش جدول‌وار آهنگ‌ها (مثلاً در Playlist View) بسیار مفید است. توجه: `setw` اندازه‌ی پنجره‌ی ترمینال را تغییر نمی‌دهد – فقط عرض ستون خروجی را کنترل می‌کند.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

// چاپ یک سطر آهنگ با ستون‌های تراز شده
void printSongRow(int index, const string& title,
                  const string& artist, int durationSec) {
    int mins = durationSec / 60;
    int secs = durationSec % 60;

    cout << left
         << setw(4) << index
         << setw(28) << title.substr(0, 27)
         << setw(16) << artist.substr(0, 15)
         << right
         << setw(2) << setfill('0') << mins << ":"
         << setw(2) << setfill('0') << secs
         << setfill(' ') << "\n";
}
```

خروجی این تابع برای سه آهنگ نمونه:

1	Bohemian Rhapsody	Queen	05:54
2	Another Brick in the Wa...	Pink Floyd	03:57
3	Hotel California	Eagles	06:30

نکات مهم: `left` و `right` جهت پر شدن فضای خالی را تعیین می‌کنند. `setfill('0')` کاراکتر پرکننده را از فاصله به صفر تغییر می‌دهد (برای نمایش زمان مثل 03:07). `setw` فقط برای یک فیلد بعدی اعمال می‌شود و بعد از چاپ آن فیلد ریست می‌شود.



۱۳ راهنمای کتابخانه‌ی miniaudio

miniaudio یک کتابخانه‌ی single-header نوشته‌شده به زبان C است که پخش و ضبط صدا را بدون هیچ وابستگی خارجی فراهم می‌کند. مثال ساده‌ی زیر نشان می‌دهد چطور یک آهنگ را با مسیر فایلش پخش کنید:

```
#include "miniaudio/miniaudio.h"

#include <stdio.h>

int main()
{
    ma_result result;
    ma_engine engine;
    ma_sound sound;

    result = ma_engine_init(NULL, &engine);
    if (result != MA_SUCCESS) {
        return -1;
    }

    result = ma_sound_init_from_file(&engine, "sound.wav", 0, NULL, NULL, &sound);
    if (result != MA_SUCCESS) {
        ma_engine_uninit(&engine);
        return -1;
    }

    ma_sound_start(&sound);

    printf("Press Enter to quit ... ");
    getchar();

    ma_sound_uninit(&sound);
    ma_engine_uninit(&engine);

    return 0;
}
```

۱۳.۱ راه‌اندازی و ساختار فایل

قبل از `#include` کردن، دقیقاً در یک فایل `.cpp` ماکرو زیر را تعریف کنید (در بقیه‌ی فایل‌ها فقط `#include` بدون ماکرو):

```
// Player.cpp - تعریف می‌شود MINIAUDIO_IMPLEMENTATION فقط اینجا
#define MINIAUDIO_IMPLEMENTATION
#include "miniaudio/miniaudio.h"
```



۱۳.۲ API های مورد نیاز

تابع	معادل در Player	توضیح
<code>ma_engine_init(NULL, &engine)</code>	constructor	راه‌اندازی موتور صوتی – یک‌بار
<code>ma_engine_uninit(&engine)</code>	destructor	آزادسازی موتور
<code>ma_sound_init_from_file(&engine, path, 0, NULL, NULL, &sound)</code>	init	بارگذاری فایل صوتی در <code>ma_sound</code>
<code>ma_sound_uninit(&sound)</code>	قبل از بارگذاری آهنگ جدید	آزادسازی sound قبلی
<code>ma_sound_start(&sound)</code>	<code>play()</code> <code>resume()</code>	شروع یا ادامه‌ی پخش (non-blocking)
<code>ma_sound_stop(&sound)</code>	<code>pause()</code>	توقف پخش؛ موقعیت حفظ می‌شود
<code>ma_sound_seek_to_pcm_frame(&sound, 0)</code>	<code>stop()</code>	بازنشانی موقعیت به ابتدا (پس از <code>ma_sound_stop</code>)
<code>ma_sound_at_end(&sound)</code>	<code>tick()</code>	بررسی پایان آهنگ – در حلقه‌ی اصلی
<code>ma_sound_get_cursor_in_pcm_frames(&sound, &frames)</code>	<code>getCurrentTime()</code>	زمان فعلی پخش به PCM frame
<code>ma_sound_get_length_in_pcm_frames(&sound, &frames)</code>	مدت کل آهنگ	طول کل به PCM frame
<code>ma_engine_get_sample_rate(&engine)</code>	تبدیل frame ↔ ثانیه	نرخ نمونه (معمولاً ۴۴۱۰۰)

۱۳.۳ تبدیل ثانیه به PCM Frame

تمام توابع seek و cursor در واحد PCM frame کار می‌کنند. تبدیل ساده است:

```
ma_uint32 sampleRate = ma_engine_get_sample_rate(&engine_);

// ثانیه → frame
ma_uint64 frame = (ma_uint64)(seconds * sampleRate);

// frame → ثانیه
float seconds = (float)frame / (float)sampleRate;
```



۱۳.۴ پیاده‌سازی seek کلیدهای جهت‌نما (±۱۰ ثانیه و امتیازی)

```
void Player::seekBy(int seconds) { // pass +10 or -10
    if (!soundLoaded_) return;

    ma_uint64 cursor, length;
    ma_sound_get_cursor_in_pcm_frames(&sound_, &cursor);
    ma_sound_get_length_in_pcm_frames(&sound_, &length);

    ma_uint32 rate = ma_engine_get_sample_rate(&engine_);
    ma_int64 newFrame = (ma_int64)cursor + (ma_int64)seconds * rate;

    if (newFrame < 0) newFrame = 0;
    if ((ma_uint64)newFrame ≥ length) { next(); return; } // past end

    ma_sound_seek_to_pcm_frame(&sound_, (ma_uint64)newFrame);
}
```

۱۳.۵ نمایش زمان جاری و تشخیص پایان آهنگ

```
// رفرش می‌شود UI در حلقه‌ی اصلی - هر بار که
void Player::tick() {
    if (state_ ≠ PlayerState::PLAYING) return;
    if (ma_sound_at_end(&sound_)) { next(); } // آهنگ تمام شد
}

float Player::getCursorSec() const {
    ma_uint64 frames = 0;
    ma_sound_get_cursor_in_pcm_frames(const_cast<ma_sound*>(&sound_), &frames);
    return (float)frames / (float)ma_engine_get_sample_rate(
        const_cast<ma_engine*>(&engine_));
}
```

⚠ یک تعریف – یک فایل

ماکرو `MINIAUDIO_IMPLEMENTATION` باید دقیقاً در یک فایل `.cpp` تعریف شود. اگر آن را در `.h` یا در چند `.cpp` قرار دهید با خطاهای «multiple definition linker» مواجه خواهید شد.

موفق و سلامت باشید